

# Password Hashing and Salting

=====

## 1. What is Hashing?

Hashing converts data (like a password) into a fixed-length random-looking string.

Example:

```
password123
↓
Hash Function
↓
a1b2c3d4e5...
```

Important properties:

- Same input → same output
- One-way process
- Cannot easily reverse the hash

---

## 2. Why Store Passwords as Hashes?

Websites should never store plain text passwords.

Bad:

```
username | password
john     | hello123
```

Good:

| username |  | hash        |
|----------|--|-------------|
| john     |  | x82jd92k... |

If the database leaks, attackers do not directly see passwords.

---

### 3. Problem Without Salt

If two users use the same password:

|       |    |          |
|-------|----|----------|
| Alice | -> | hello123 |
| Bob   | -> | hello123 |

Then both hashes will also be the same.

hash(hello123)  
= SAME HASH

This creates security risks.

Attackers can:

- detect repeated passwords
  - crack one hash and access multiple accounts
- 

### 4. What is Salt?

A salt is a random string added to the password before hashing.

password + salt

↓

hash

Example:

Password = hello123

Salt = Xy91@k

Final Input:

hello123Xy91@k

---

## 5. Why is Salt Important?

Salt makes hashes unique.

Example:

**Alice**

Password = hello123

Salt = abc111

hash(hello123abc111)

= HASH1

## Bob

Password = hello123

Salt = xyz999

hash(hello123xyz999)

= HASH2

Same password, different hashes.

---

## 6. How Salt Protects Against Attacks

Without salt:

- attackers can use precomputed tables
- rainbow table attacks become easy

With salt:

- every user has a different hash
  - attackers must crack each password separately
  - attacks become much slower
- 

## 7. Is Salt Secret?

No.

Salt is usually stored in the database along with the hash.

Example:

| Username | Salt   | Hash     |
|----------|--------|----------|
| john     | Xy91@k | abcd1234 |

The salt is needed during login verification.

---

## 8. Login Process

### During Registration

```
graph TD; A[Password] --> B[Generate random salt]; B --> C[password + salt]; C --> D[Hash]; D --> E[Store hash + salt];
```

---

### During Login

```
graph TD; A[User enters password] --> B[Fetch stored salt]; B --> C[password + salt]; C --> D[Hash again]; D --> E[Compare hashes];
```

If hashes match:

- login successful
- 

## 9. What Makes a Good Salt?

A good salt should be:

- Random
- Unique for every user
- Cryptographically secure
- Long enough

Good sources:

- SecureRandom
- CSPRNG

Bad sources:

- predictable words
  - Math.random()
- 

## 10. System-Wide Salt vs Unique Salt

### Bad

Same salt for everyone:

password + COMMON\_SALT

Still produces matching hashes for same passwords.

---

## Good

Different salt per user.

---

## 11. bcrypt

bcrypt is a password hashing algorithm.

It:

- automatically uses salt
- is intentionally slow
- makes brute-force attacks difficult

Modern password hashing algorithms:

- bcrypt
- Argon2
- scrypt

Weak/old algorithms:

- MD5
  - SHA1
- 

## 12. Why Slow Hashing is Good

Fast hashing is dangerous for passwords.

Attackers can try billions of guesses per second.

bcrypt is intentionally slow, so password cracking becomes expensive and time-consuming.

---

## 13. Salt vs Pepper

### Salt

- unique per user
- stored in database
- not secret

### Pepper

- secret value stored on server
- adds extra security

Example:

```
hash(password + salt + pepper)
```

---

## 14. Important Security Point

Even if passwords are salted:

- companies should assume passwords may still be compromised after a breach
  - users should reset passwords immediately
- 

## 15. Simple Interview Summary

### Hashing



Converts passwords into fixed random strings.

## Salt

Random data added before hashing.

## Why Use Salt?

- prevents identical hashes
- stops rainbow table attacks
- increases attack cost

## Best Practice

```
bcrypt(password + unique_random_salt)
```

---

## 16. One-Line Summary

Salt does not make passwords stronger.

Salt makes password hashes unique and prevents large-scale precomputed attacks.

Sources based on Auth0, Proton, and GeeksforGeeks explanations.